

Buổi 3 — Design pattern

Chủ đề: Observer, Adapter

1. Định nghĩa

❖ Observer

- Định nghĩa mối phụ thuộc một - nhiều giữa các đối tượng để khi mà một đối tượng có sự thay đổi trạng thái, tất cả các thành phần phụ thuộc của nó sẽ được thông báo và cập nhật một cách tự động.
- Một đối tượng có thể thông báo đến một số lượng không giới hạn các đối tượng khác

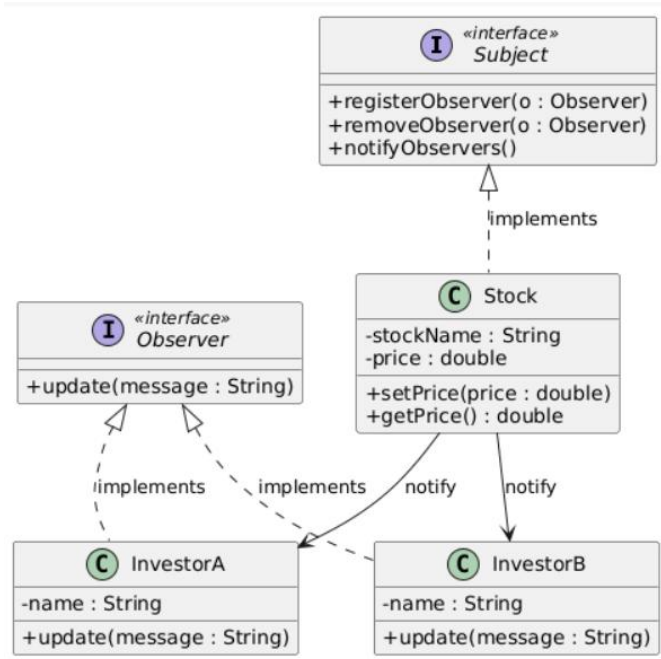
❖ Adapter

- Adapter Pattern là mẫu thiết kế cấu trúc (*Structural Pattern*), cho phép hai interface không tương thích có thể làm việc với nhau thông qua một lớp trung gian gọi là Adapter.

2. Bài tập

❖ Observer

Khi giá của một cổ phiếu thay đổi, các nhà đầu tư đã đăng ký để theo dõi cổ phiếu đó sẽ nhận thông báo ngay lập tức về sự thay đổi.



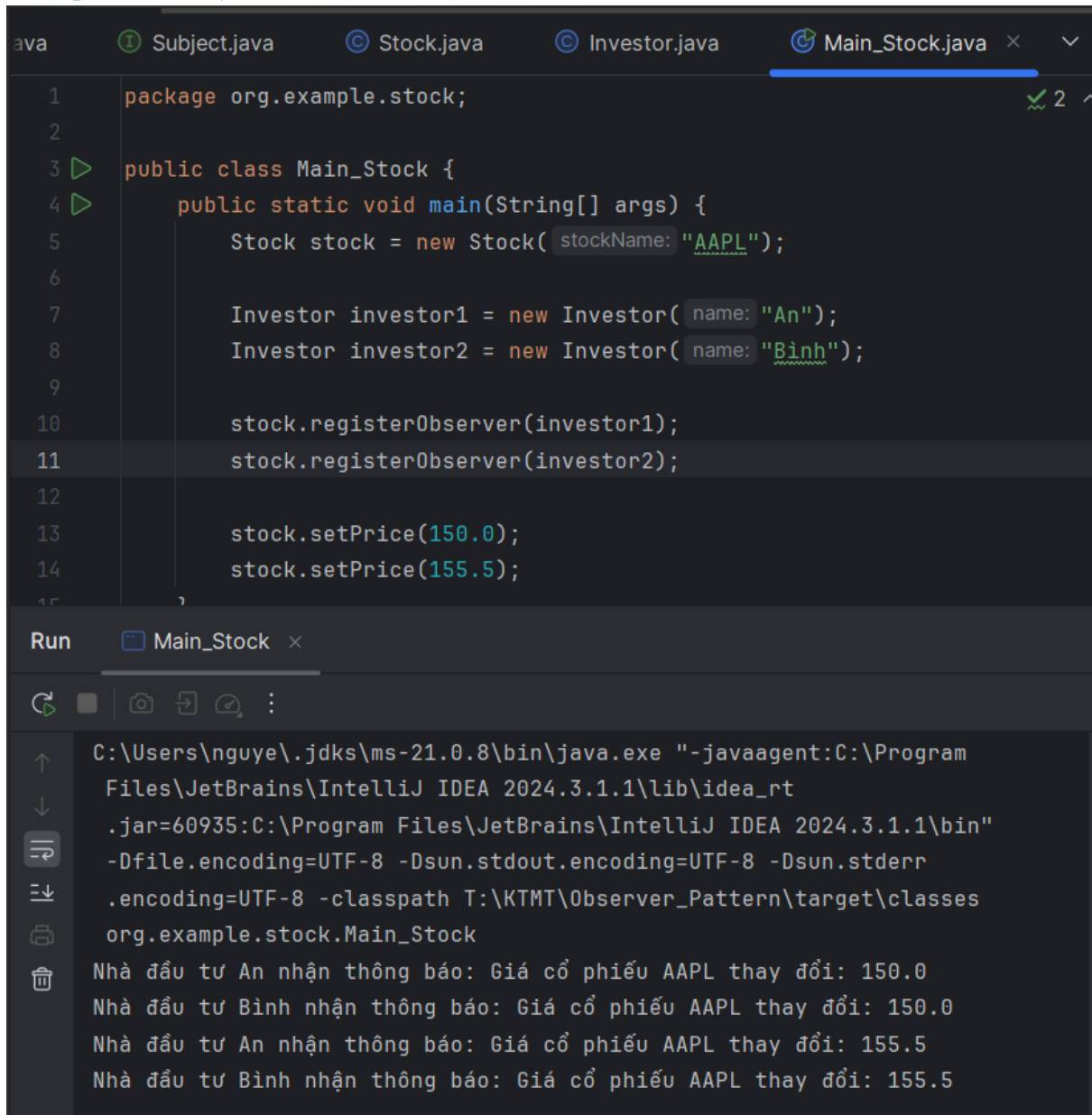
```
Observer.java Subject.java x Stock.java Investor.java Main_Stock.java
1 package org.example;
2
3 public interface Subject { no usages 1 implementation
4     void registerObserver(Observer o); no usages 1 implementation
5     void removeObserver(Observer o); no usages 1 implementation
6     void notifyObservers(); no usages 1 implementation
7 }
```

```
Observer.java x Subject.java Stock.java Investor.java Main_Stock.java
1 package org.example;
2
3 public interface Observer { no usages 1 implementation
4     void update(String message); no usages 1 implementation
5 }
6
```

```
Observer.java Subject.java Stock.java x Investor.java Main_ v ...
8
9 public class Stock implements Subject { no usages
10
11     private String stockName; 2 usages
12     private double price; 3 usages
13     private List<Observer> observers = new ArrayList<>(); 3 usages
14
15     public Stock(String stockName) { no usages
16         this.stockName = stockName;
17     }
18
19     public void setPrice(double price) { no usages
20         this.price = price;
21         notifyObservers();
22     }
23
24     public double getPrice() { no usages
25         return price;
26     }
27
28     @Override no usages
29     public void registerObserver(Observer o) {
30         observers.add(o);
31     }
32
33     @Override no usages
34     public void removeObserver(Observer o) {
35         observers.remove(o);
```

```
Project Alt+1 a Subject.java Stock.java Investor.java x Main_ v ...
1 package org.example.stock;
2
3 import org.example.Observer;
4
5 public class Investor implements Observer { 4 usages
6
7     private String name; 2 usages
8
9     public Investor(String name) { 2 usages
10         this.name = name;
11     }
12
13     @Override 1 usage
14     public void update(String message) {
15         System.out.println("Nhà đầu tư " + name + " nhận thông báo: "
16             + message);
17     }
18 }
19
```

Kết quả khi chạy



The screenshot displays an IDE with a Java project. The top pane shows the code for `Main_Stock.java`, which implements the Observer pattern. It creates a `Stock` object for AAPL, registers two `Investor` objects (An and Binh), and updates the stock price from 150.0 to 155.5. The bottom pane shows the command prompt output, which displays the price change notifications for both investors.

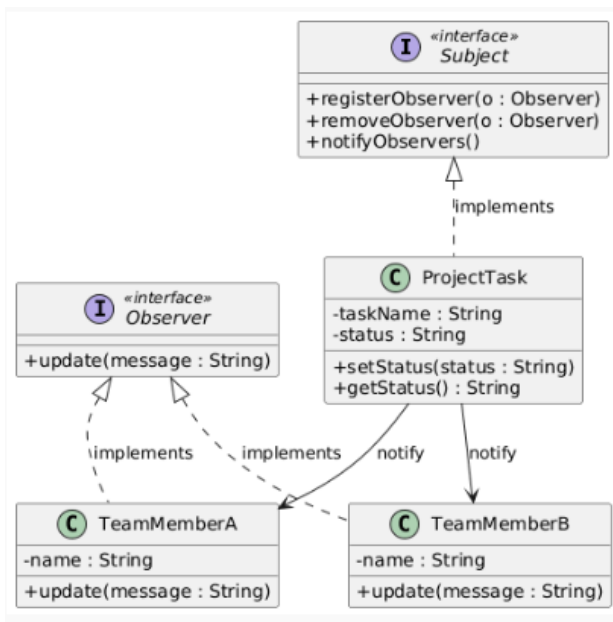
```
1 package org.example.stock;
2
3 public class Main_Stock {
4     public static void main(String[] args) {
5         Stock stock = new Stock( stockName: "AAPL");
6
7         Investor investor1 = new Investor( name: "An");
8         Investor investor2 = new Investor( name: "Binh");
9
10        stock.registerObserver(investor1);
11        stock.registerObserver(investor2);
12
13        stock.setPrice(150.0);
14        stock.setPrice(155.5);
15    }
16 }
```

Run Main_Stock

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\lib\idea_rt.jar=60935:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\Observer_Pattern\target\classes org.example.stock.Main_Stock

Nhà đầu tư An nhận thông báo: Giá cổ phiếu AAPL thay đổi: 150.0
Nhà đầu tư Binh nhận thông báo: Giá cổ phiếu AAPL thay đổi: 150.0
Nhà đầu tư An nhận thông báo: Giá cổ phiếu AAPL thay đổi: 155.5
Nhà đầu tư Binh nhận thông báo: Giá cổ phiếu AAPL thay đổi: 155.5

Trong một dự án phần mềm, khi có sự thay đổi về tình trạng hoặc trạng thái công việc (task), các thành viên trong nhóm sẽ nhận được thông báo tự động để theo dõi tiến độ.



```

Main_Stock.java x  ProjectTask.java x  TeamMember.java  Main_Task.java v
9      public class ProjectTask implements Subject { no usages  3 1
11      private String taskName; 2 usages
12      private String status; 3 usages
13      private List<Observer> observers = new ArrayList<>(); 3 usages
14
15      public ProjectTask(String taskName) { no usages
16      |      this.taskName = taskName;
17      |
18      |
19      public void setStatus(String status) { no usages
20      |      this.status = status;
21      |      notifyObservers();
22      |
23
24      public String getStatus() { no usages
25      |      return status;
26      |
27      |
28      @Override no usages
29      public void registerObserver(Observer o) {
30      |      observers.add(o);
31      |
32      |
33      @Override no usages
34      public void removeObserver(Observer o) {
35      |      observers.remove(o);
36      |
  
```

```
1 package org.example.task;
2
3 import org.example.Observer;
4
5 public class TeamMember implements Observer { no usages
6
7     private String name; 2 usages
8
9     public TeamMember(String name) { no usages
10         this.name = name;
11     }
12
13     @Override 2 usages
14     public void update(String message) {
15         System.out.println("Thành viên " + name + " nhận thông báo: " + message);
16     }
17 }
```

Kết quả khi chạy

```
1 package org.example.task;
2
3 public class Main_Task {
4     public static void main(String[] args) {
5         ProjectTask task = new ProjectTask(taskName: "Thiết kế hệ thống");
6
7         TeamMember member1 = new TeamMember(name: "Lan");
8         TeamMember member2 = new TeamMember(name: "Minh");
9
10        task.registerObserver(member1);
11        task.registerObserver(member2);
12
13        task.setStatus("In Progress");
14        task.setStatus("Completed");
15    }
16 }
```

Debug Main_Task

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe ...

Connected to the target VM, address: '127.0.0.1:61395', transport: 'socket'

Thành viên Lan nhận thông báo: Task 'Thiết kế hệ thống' đổi trạng thái: In Progress

Thành viên Minh nhận thông báo: Task 'Thiết kế hệ thống' đổi trạng thái: In Progress

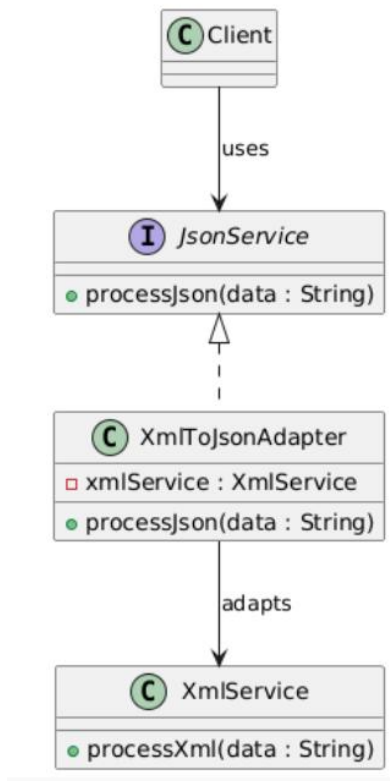
Thành viên Lan nhận thông báo: Task 'Thiết kế hệ thống' đổi trạng thái: Completed

Thành viên Minh nhận thông báo: Task 'Thiết kế hệ thống' đổi trạng thái: Completed

Disconnected from the target VM, address: '127.0.0.1:61395', transport: 'socket'

❖ Adapter

Một dịch vụ web yêu cầu đầu vào ở định dạng JSON, nhưng một hệ thống khác chỉ hỗ trợ XML. Bạn có thể viết một adapter để chuyển đổi dữ liệu từ XML sang JSON và ngược lại.



```
1 package org.example.adapter;
2
3 public interface JsonService {
4     void processJson(String jsonData);
5 }
```

```
1 package org.example.adapter;
2
3 public class XmlService {
4
5     public void processXml(String xmlData) {
6         System.out.println("Xử lý dữ liệu XML: " + xmlData);
7     }
8 }
9
```

```
1 package org.example.adapter;
2
3 public class XmlToJsonAdapter implements JsonService { no usages
4
5     private XmlService xmlService; 2 usages
6
7     public XmlToJsonAdapter(XmlService xmlService) { no usages
8         this.xmlService = xmlService;
9     }
10
11     @Override no usages
12     public void processJson(String jsonData) {
13         // Giả lập chuyển JSON → XML
14         String xmlData = "<data>" + jsonData + "</data>";
15         xmlService.processXml(xmlData);
16     }
17 }
```

Kết quả chạy

```
1 package org.example.adapter;
2
3 public class Main_Adapter {
4     public static void main(String[] args) {
5
6         XmlService xmlService = new XmlService();
7
8         // Adapter chuyển XML ↔ JSON
9         JsonService adapter = new XmlToJsonAdapter(xmlService);
10
11         // Client chỉ biết JSON
12         String json = "{ \"name\": \"ChatGPT\", \"type\": \"AI\" }";
13         adapter.processJson(json);
14     }
15 }
16
```

Run Main_Adapter

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1\lib\idea_rt.jar=52615:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1\bin" -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\Observer_Pattern\target\classes org.example.adapter.Main_Adapter

Xử lý dữ liệu XML: <data>{ "name": "ChatGPT", "type": "AI" }</data>

Process finished with exit code 0